



jQuery & AJAX

Study Notes — Framework · Events · UI Components · AJAX

jQuery basics

Selectors & Events

jQuery UI widgets

AJAX & JSON

Runnable code examples

Theory · Real-world example · Code, for every concept

📄 Contents

Four topics, from jQuery basics to talking to a server with AJAX.

01	📖 jQuery Framework	10 concepts
02	📖 jQuery Events	12 concepts
03	📖 jQuery UI Components	15 concepts
04	🌐 AJAX: Data Exchange with Server	12 concepts

01 ❤️ jQuery Framework

Read the theory, look at the code, and try the concepts yourself.

1. What is jQuery?

jQuery is a **JavaScript library**. It helps us write **less code** to do more work.

The **\$** sign means jQuery.

The basic rule is: `$("#find this").doThis()`

🔗 Plain JS: `document.getElementById("hello").textContent = "Hi"` → jQuery: `$("#hello").text("Hi")`

CODE

```
function whatIsjQuery() {  
  // $( ) is the jQuery function. It finds an element on the page.  
  // "#hello" means: find the element with id="hello"  
  // .text("...") is a jQuery method. It changes the text inside.  
  $("#hello").text("Hello! jQuery changed this text 🤖");  
}
```

2. Selectors

A **selector** tells jQuery **which element** to work on.

`#name` → finds by **id** (only one element).

`.name` → finds by **class** (can be many elements).

`p`, `b`, `h1` → finds by **tag**.

🔗 Like calling students: by roll number (#id), by group (.class), or "everyone" (tag).

CODE

```
function selectors() {  
  // "#first" -> # means id. Finds the ONE element with id="first"  
  // .css("color", "red") -> changes its text color to red  
  $("#first").css("color", "red");  
  
  // ".fruit" -> . (dot) means class. Finds ALL elements with class="fruit"  
  $(".fruit").css("background", "yellow");  
  
  // "#demo-selectors b" -> finds all <b> tags inside #demo-selectors  
  $("#demo-selectors b").css("font-size", "22px");  
}
```

3. Events

An **event** is something the user does, like a **click**.

We tell jQuery: **when this happens, do that**.

🔗 Like a doorbell: press it (event) → it rings (action).

CODE

```
function events() {  
  // .off("click") removes any old click action  
  // (so clicking Run many times doesn't add many actions)  
  $("#hiBtn").off("click");  
  
  // .click( function(){ ... } ) means:  
  // "when this button is clicked, run the code inside { }"  
  $("#hiBtn").click(function () {  
    $("#hiMsg").text("You clicked the button! 🤖");  
  });  
}
```

4. Hide, Show and Toggle

`hide()` → makes it **disappear**.

`show()` → makes it **appear**.

`toggle()` → **switches** between the two.

💡 *toggle() is like a light switch: press once ON, press again OFF.*

CODE

```
function hideShow() {
  // .hide(1000) -> hides the box slowly (1000 ms = 1 second)
  $("#box1").hide(1000);
  // .show(1000) -> shows the box again slowly
  $("#box1").show(1000);
}

function toggleDemo() {
  // .toggle() -> if visible, hide it. If hidden, show it.
  $("#box2").toggle(500);
}
```

5. Fade and Slide

Fade → slowly disappears or appears.

Slide → rolls up or rolls down.

These make the page look **smooth**.

💡 *Fade is like a movie scene going dark. Slide is like a shop shutter.*

CODE

```
function fadeSlide() {
  // .fadeOut() -> slowly becomes invisible
  // .fadeIn() -> slowly becomes visible
  $("#box3").fadeOut(800).fadeIn(800);

  // .slideUp() -> rolls up and hides
  // .slideDown() -> rolls down and shows
  $("#box4").slideUp(800).slideDown(800);
}
```

6. Get and Set Content

Get = read what is inside. **Set** = change what is inside.

`text()` → plain text. `html()` → text with tags. `val()` → input box value.

Empty brackets → **get**. Something inside → **set**.

💡 *val() reads what you typed. text("Hi") writes "Hi" on the page.*

CODE

```
function getSetContent() {
  // .val() with empty brackets -> GETS what the user typed in the input
  let name = $("#nameInput").val();

  // .text(...) with something inside -> SETS the text
  $("#greet").text("Welcome, " + name + "!");

  // .html(...) -> SETS content WITH HTML tags (here <b> for bold)
  $("#bold").html("<b>This is bold using html()</b>");
}
```

7. CSS and Classes

`css()` → changes **one style** (like color or border).

`toggleClass()` → adds or removes a **whole set of styles** at once.

👉 *`css()` = changing only your shirt. `toggleClass()` = changing into a full uniform.*

CODE

```
function cssClasses() {
  // .css("property", "value") -> changes one style
  $("#styleMe").css("border", "3px solid green");

  // .toggleClass("highlight") -> adds the class "highlight"
  // Run again -> removes it
  $("#styleMe").toggleClass("highlight");
}
```

8. Add and Remove Elements

`append()` → **adds** something new at the end.

`remove()` → **deletes** an element from the page.

👉 *Like a shopping list: write a new item at the bottom, or cross one out.*

CODE

```
function addElement() {
  // .append(...) -> adds new HTML at the END inside #myList
  $("#myList").append("<li>New item ✨</li>");
}

function removeElement() {
  // "#myList li" -> all <li> items inside #myList
  // .last()      -> pick only the last one
  // .remove()    -> delete it from the page
  $("#myList li").last().remove();
}
```

9. Animate

`animate({ style: value }, time)` → makes your own animation.

You tell it **what to change** (like width) and **how long** it takes.

👉 *Like stretching a rubber band and letting it come back.*

CODE

```
function animateDemo() {
  $("#box5").animate({ width: "300px" }, 1000); // grow wider
  $("#box5").animate({ width: "120px" }, 1000); // back to normal
}
```

10. Chaining

Chaining = doing **many actions** on the same element, joined with dots.

Select once, then keep adding actions. **Less code!**

👉 *"Go to shop, buy milk, come back" said as one sentence.*

CODE

```
function chaining() {
  // Select the box ONCE, then add many actions with dots.
  // They run one after another, from top to bottom.
  $("#box6")
    .css("background", "purple") // 1. change color
    .slideUp(600)                // 2. roll up
    .slideDown(600);             // 3. roll down
}
```

02 📁 jQuery Events

Read the theory, look at the code, then do the action described in each example.

1. What is an Event? + ready()

An **event** is something the user does on the page.

Examples: **click**, **typing**, **moving the mouse**, **sending a form**.

`$(document).ready()` → wait for the page to load, then run the code.

👉 Like a teacher waiting for all students to sit before starting class.

CODE

```
function readyDemo() {  
  // $(document) -> the whole web page  
  // .ready( function(){ ... } ) -> run the code inside { }  
  // ONLY after the page has fully loaded  
  $(document).ready(function () {  
    $("#readyMsg").text("Page is loaded. Now jQuery code can run 🎉");  
  });  
}
```

2. click()

`click()` → runs when the element is **clicked once**.

This is the **most used** event.

👉 Pressing a lift button.

CODE

```
function clickDemo() {  
  $("#clickBtn").off("click");  
  $("#clickBtn").click(function () {  
    $("#clickMsg").text("You clicked! 🎉");  
  });  
}
```

3. dblclick()

`dblclick()` → runs when clicked **two times quickly**.

👉 Double-clicking a folder to open it.

CODE

```
function dblclickDemo() {  
  $("#dblBox").off("dblclick");  
  // .dblclick() -> runs only when clicked TWO times quickly  
  $("#dblBox").dblclick(function () {  
    $("#dblBox").text("Double clicked! 🎉");  
  });  
}
```

4. mouseenter() and mouseleave()

`mouseenter()` → mouse comes **onto** the element.

`mouseleave()` → mouse goes **away** from the element.

💡 Automatic door: opens when you come near, closes when you leave.

CODE

```
function mouseDemo() {
  $("#mouseBox").off("mouseenter mouseleave");

  $("#mouseBox").mouseenter(function () {
    $("#mouseBox").css("background", "orange").text("Mouse IN 🐭");
  });

  $("#mouseBox").mouseleave(function () {
    $("#mouseBox").css("background", "#4a9fd8").text("Mouse OUT");
  });
}
```

5. hover()

`hover()` = `mouseenter` + `mouseleave` in **one method**.

It takes **2 functions**: first for IN, second for OUT.

💡 A button that changes color when your mouse is on it.

CODE

```
function hoverDemo() {
  $("#hoverBox").off("mouseenter mouseleave");
  // .hover( function 1, function 2 )
  $("#hoverBox").hover(
    function () { $("#hoverBox").css("background", "green"); },
    function () { $("#hoverBox").css("background", "#4a9fd8"); }
  );
}
```

6. keyup() — typing

`keyup()` → runs when you **release a key** while typing.

Useful for **live search** or **live preview**.

💡 Google showing suggestions as you type.

CODE

```
function keyupDemo() {
  $("#typeInput").off("keyup");
  $("#typeInput").keyup(function () {
    let typed = $("#typeInput").val();
    $("#typeMsg").text("You typed: " + typed);
  });
}
```

7. focus() and blur()

`focus()` → you click **inside** an input box.

`blur()` → you **leave** the input box.

☞ A form box that glows when you are typing in it.

CODE

```
function focusBlurDemo() {
  $("#focusInput").off("focus blur");

  $("#focusInput").focus(function () {
    $("#focusInput").css("background", "#fff3b0");
    $("#focusMsg").text("You are inside the box 🟡");
  });

  $("#focusInput").blur(function () {
    $("#focusInput").css("background", "white");
    $("#focusMsg").text("You left the box 🟡");
  });
}
```

8. change()

`change()` → runs when a **value is changed**.

Used with **dropdowns**, **checkboxes** and **radio buttons**.

☞ Picking your state in a form, then the city list updates.

CODE

```
function changeDemo() {
  $("#citySelect").off("change");
  $("#citySelect").change(function () {
    let city = $("#citySelect").val();
    $("#changeMsg").text("You chose: " + city);
  });
}
```

9. submit()

`submit()` → runs when a **form is sent**.

`e.preventDefault()` → stops the page from **reloading**.

☞ Pressing "Login" on a website.

CODE

```
function submitDemo() {
  $("#myForm").off("submit");
  // "e" is the event object (details about the event)
  $("#myForm").submit(function (e) {
    e.preventDefault(); // STOPS the page from reloading
    let name = $("#formName").val();
    $("#submitMsg").text("Form sent! Hello " + name + " 📧");
  });
}
```

10. on() — the modern way

`on("event", function)` → adds **any** event.

Same work as `click()`, `hover()` etc., but **one method for all**.

Most new jQuery code uses **on()**.

☞ *One universal remote instead of many remotes.*

CODE

```
function onDemo() {
    $("#onBox").off();    // .off() with nothing -> remove ALL events

    $("#onBox").on("click", function () {
        $("#onMsg").text("click happened");
    });
    $("#onBox").on("dblclick", function () {
        $("#onMsg").text("dblclick happened");
    });
    $("#onBox").on("mouseleave", function () {
        $("#onMsg").text("mouseleave happened");
    });
}
```

11. Event Object (e)

When an event happens, jQuery gives us an **event object**, usually named `e`.

It has **details** about the event: what happened, where the mouse was, etc.

☞ *Like a CCTV record: what happened, and where.*

CODE

```
function eventObjectDemo() {
    $("#objBox").off("click");
    $("#objBox").click(function (e) {
        // e.type -> name of the event
        // e.pageX -> mouse position from the left
        // e.pageY -> mouse position from the top
        $("#objMsg").text(
            "Event: " + e.type + " | X: " + e.pageX + " | Y: " + e.pageY
        );
    });
}
```

12. off() — remove an event

`off()` → **removes** an event.

After that, the action **stops working**.

☞ *Unplugging a doorbell: pressing it does nothing now.*

CODE

```
function addOffEvent() {
    $("#offBtn").off("click");
    $("#offBtn").click(function () {
        $("#offMsg").text("Button is working ☺");
    });
}

function removeOffEvent() {
    // .off("click") -> removes the click event
    $("#offBtn").off("click");
    $("#offMsg").text("Event removed. Click the button now, nothing happens ☹");
}
```

03 🛠️ jQuery UI Components

jQuery UI is an add-on that gives ready-made parts for a website. It needs two extra files: its JS and its CSS.

1. What is jQuery UI?

jQuery UI is an **add-on** for jQuery.

It gives **ready-made parts** for a website, like calendars, popups and tabs.

You add **2 extra lines** in HTML: jQuery UI's **JS file** and its **CSS file**.

Rule: `$("#id").componentName()` → turns a normal element into a UI component.

💡 Like buying ready-made furniture instead of building it from wood.

CODE

```
function introDemo() {  
  // $.ui.version -> tells which version of jQuery UI is loaded  
  $("#introMsg").text("jQuery UI is loaded ✓ Version: " + $.ui.version);  
}
```

2. Datepicker

`datepicker()` → shows a **calendar** to pick a date.

The user clicks a date instead of typing it.

💡 Choosing your date of birth on a form.

CODE

```
function datepickerDemo() {  
  // .datepicker() -> turns a normal input box into a calendar box  
  $("#dateInput").datepicker();  
}
```

3. Accordion

`accordion()` → sections that **open and close** on click.

Only **one section** is open at a time.

💡 An FAQ page: click a question to see its answer.

CODE

```
function accordionDemo() {  
  // .accordion() -> turns headings + divs into open/close sections  
  $("#myAccordion").accordion();  
}
```

4. Tabs

`tabs()` → shows content in **tabs**.

Click a tab to see **only its content**.

💡 Tabs in your web browser.

CODE

```
function tabsDemo() {  
  // .tabs() -> turns a list of links + divs into clickable tabs  
  // Each link (#tab1) opens the div with that id  
  $("#myTabs").tabs();  
}
```

5. Dialog

`dialog()` → shows a **popup box** on the page.

It can be **moved, resized** and **closed**.

🔗 *"Are you sure you want to delete?" popup.*

CODE

```
function dialogDemo() {
  // .dialog({ ... }) -> turns a div into a popup box
  // modal: true -> dark background, user must close the popup first
  $("#myDialog").dialog({
    title: "Hello!",
    modal: true
  });
}
```

6. Autocomplete

`autocomplete()` → shows **suggestions** while you type.

The suggestions come from a **list** we give it.

🔗 *Google search suggestions.*

CODE

```
function autocompleteDemo() {
  let languages = ["C", "C++", "Java", "JavaScript", "Python", "PHP", "Ruby"];
  // .autocomplete({ source: list }) -> shows suggestions while typing
  $("#langInput").autocomplete({
    source: languages
  });
}
```

7. Slider

`slider()` → a bar you **drag** to pick a number.

We set the **min, max** and **starting value**.

🔗 *Volume or brightness control on your phone.*

CODE

```
function sliderDemo() {
  $("#mySlider").slider({
    min: 0,
    max: 100,
    value: 50,
    // slide: -> runs every time the slider moves
    slide: function (event, ui) {
      $("#sliderMsg").text("Volume: " + ui.value);
    }
  });
}
```

8. Progressbar

`progressbar()` → shows **how much work is done**.

Value goes from **0 to 100** (percent).

🔗 *Download or file upload bar.*

CODE

```
function startProgress() {
  $("#myBar").progressbar({ value: 0 });
}

function addProgress() {
  // GET the current value, then SET a new one
  let now = $("#myBar").progressbar("value");
  $("#myBar").progressbar("value", now + 10);
}
```

9. Spinner

`spinner()` → a number box with **up ▲ and down ▼ arrows**.

We can set **min** and **max** limits.

☞ Choosing quantity in a shopping cart.

CODE

```
function spinnerDemo() {  
  // .spinner() -> adds up/down arrows to a number box  
  $("#qtyInput").spinner({  
    min: 1,  
    max: 10  
  });  
}
```

10. Tooltip

`tooltip()` → shows a **small hint** when the mouse is on something.

The hint text comes from the `title` attribute.

☞ Hovering on an icon and seeing its name.

CODE

```
function tooltipDemo() {  
  // .tooltip() -> shows a small hint box when the mouse is on an element  
  $("#tipArea").tooltip();  
}
```

11. Draggable

`draggable()` → you can **pick and move** the element with the mouse.

☞ Moving app icons on your phone screen.

CODE

```
function draggableDemo() {  
  $("#dragBox").draggable();  
}
```

12. Droppable

`droppable()` → an area where you can **drop** a dragged item.

We can run code **when the drop happens**.

☞ Dragging a file into the Recycle Bin.

CODE

```
function droppableDemo() {  
  $("#dropItem").draggable();  
  // .droppable({ ... }) -> makes an area that ACCEPTS dropped items  
  $("#dropZone").droppable({  
    drop: function () {  
      $("#dropZone").css("background", "#b9f6ca").text("Dropped! ☑");  
    }  
  });  
}
```

13. Resizable

`resizable()` → you can **change the size** by pulling the corner.

☞ Making a window bigger or smaller on your computer.

CODE

```
function resizableDemo() {  
  $("#sizeBox").resizable();  
}
```

14. Sortable

`sortable()` → drag list items to **change their order**.

🔗 *Rearranging songs in a playlist.*

CODE

```
function sortableDemo() {  
  // .sortable() -> lets you drag list items up and down to reorder them  
  $("#sortList").sortable();  
}
```

15. Selectable

`selectable()` → **click to select** items from a list.

Hold **Ctrl** to select **many**.

🔗 *Selecting many photos in your gallery.*

CODE

```
function selectableDemo() {  
  // .selectable() -> click (or drag over) items to select them  
  $("#selectList").selectable();  
}
```



AJAX = Asynchronous JavaScript And XML. It lets a page talk to the server in the background, without reloading.

1. What is AJAX?

AJAX = **A**ynchronous **J**avaScript **A**nd **X**ML.

It lets a web page **talk to the server** in the background.

The page gets new data **without reloading**.

Asynchronous = the page does not wait. You can keep using it while data comes.

Today we mostly use **JSON** instead of XML.

🔗 Like Instagram loads more posts as you scroll. The page never reloads.

1. You click → 2. Request to server → 3. Server sends data → 4. Page updates (no reload)

2. JSON — the data format

JSON is the most common way to **send data** between page and server.

It looks like: `{"name": "Ravi", "age": 20}`

`JSON.stringify()` → object to text (for **sending**).

`JSON.parse()` → text to object (for **using**).

🔗 Like packing things in a box to courier them, then unpacking at the other end.

CODE

```
function jsonDemo() {
  let student = { name: "Ravi", age: 20 };

  // JSON.stringify() -> object → JSON text (to SEND to server)
  let text = JSON.stringify(student);

  // JSON.parse() -> JSON text → object (to USE data from server)
  let obj = JSON.parse(text);

  $("#jsonMsg").text("As JSON text: " + text + " | name = " + obj.name);
}
```

3. XMLHttpRequest — the old way

`XMLHttpRequest` is the **original** way to do AJAX in plain JavaScript.

Steps: **create** → **open** → **onload** → **send**.

Good to know for exams. Longer to write.

🔗 Like writing a letter by hand and posting it yourself.

CODE

```
function xhrDemo() {
  let xhr = new XMLHttpRequest();
  // .open("GET", url) -> say WHAT we want and FROM WHERE
  xhr.open("GET", SERVER + "/users/1");

  // .onload -> runs when the server's reply arrives
  xhr.onload = function () {
    let user = JSON.parse(xhr.responseText);
    document.getElementById("xhrMsg").textContent = "Name: " + user.name;
  };

  xhr.send(); // actually send the request
}
```

4. fetch() — the modern way

`fetch()` is the **new, shorter** way in plain JavaScript.

`.then()` means: **after this finishes, do this next**.

☞ Like ordering food online: order → wait → food arrives → eat.

CODE

```
function fetchDemo() {
  fetch(SERVER + "/users/2")
    .then(function (response) {
      return response.json(); // convert reply into an object
    })
    .then(function (user) {
      $("#fetchMsg").text("Name: " + user.name + " | Email: " + user.email);
    });
}
```

5. \$.ajax() — jQuery full method

`$.ajax()` is jQuery's **main** AJAX method.

We give settings like **url**, **type** and **success**.

It can do **everything**: get, send, show errors.

☞ A full order form where you choose every detail.

CODE

```
function ajaxDemo() {
  $.ajax({
    url: SERVER + "/users/3",
    type: "GET",
    success: function (data) {
      $("#ajaxMsg").text("Name: " + data.name + " | City: " + data.address.city);
    }
  });
}
```

6. \$.get() — get data

`$.get()` → a **short way** to **get** data from the server.

Just give the **url** and a **function**.

☞ Asking the library: "Give me book number 1."

CODE

```
function getDemo() {
  $.get(SERVER + "/posts/1", function (data) {
    $("#getMsg").text("Post title: " + data.title);
  })
  .fail(function (xhr) {
    $("#getMsg").text("✖ Could not get data. Error code: " + xhr.status);
  });
}
```

7. \$.getJSON() — get JSON data

`$.getJSON()` → same as `$.get()`, but made **only for JSON** data.

☞ Same library request, but you say "only give me it in English".

CODE

```
function getJSONDemo() {
  $.getJSON(SERVER + "/todos/1", function (data) {
    $("#getJSONMsg").text("Task: " + data.title + " | Done? " + data.completed);
  });
}
```

8. \$.post() — send data

`$.post()` → **sends** data to the server.

GET = take data. **POST** = give data.

Used for **forms**, like signup or comments.

📝 *Submitting your exam form to the college office.*

CODE

```
function postDemo() {
  let newPost = {
    title: $("#postTitle").val(),
    body: "Sent from jQuery",
    userId: 1
  };

  $.post(SERVER + "/posts", newPost, function (reply) {
    $("#postMsg").text("Saved ✓ New id: " + reply.id + " | Title: " + reply.title);
  });
}
```

9. load() — put data straight into the page

`load()` → gets data and puts it **directly inside** an element.

The **shortest** AJAX method. Best for loading **HTML parts**.

📝 *Like pasting a page from another notebook directly into yours.*

CODE

```
function loadDemo() {
  // .load(url, function) -> gets data and puts it straight INSIDE the element
  $("#loadArea").load(SERVER + "/posts/2", function (response, status, xhr) {
    if (status == "error") {
      $("#loadArea").text("✗ Could not load. Error code: " + xhr.status);
    }
  });
}
```

10. Showing many records

Servers often send a **list** (array) of items.

`$.each()` → goes through **each item** one by one.

We **append** each item to the page.

📝 *Showing all students' names from the college database.*

CODE

```
function manyRecordsDemo() {
  $.getJSON(SERVER + "/users", function (users) {
    $("#userList").empty(); // .empty() -> clear old items

    // $.each(list, function(index, item)) -> loop over every item
    $.each(users, function (i, user) {
      $("#userList").append("<li>" + user.name + " — " + user.email + "</li>");
    });
  });
}
```

11. Error handling

Sometimes the request **fails** (wrong address, no internet, server down).

`error:` → runs when it fails, so we can **show a message**.

404 = not found. **500** = server problem.

☞ Calling a wrong number and hearing "This number does not exist".

CODE

```
function errorDemo() {
  $.ajax({
    url: SERVER + "/wrong-address",    // this address does NOT exist
    success: function () {
      $("#errorMsg").text("Worked!");
    },
    // error: -> runs when the request FAILS
    error: function (xhr) {
      $("#errorMsg").text("✖ Something went wrong! Error code: " + xhr.status);
    }
  });
}
```

12. Loading message

`beforeSend:` → runs **before** the request (show "Loading...").

`complete:` → runs at the **end**, whether it worked or failed.

☞ The spinning circle you see before a video starts.

CODE

```
function loadingDemo() {
  $.ajax({
    url: SERVER + "/photos/1",
    beforeSend: function () {
      $("#loadingMsg").text("Loading... 🔄");
    },
    success: function (data) {
      $("#loadingMsg").text("Photo title: " + data.title);
    },
    complete: function () {
      $("#loadingDone").text("Request finished 🏁");
    }
  });
}
```